REPUBLIQUE DU TCHAD
Unité – Travail – Progrès
****
MINISTERE DE L'ENSEIGNEMENT SUPERIEUR
****
UNIVERSITE DE MOUNDOU
****
FACULTE DES SCIENCES EXACTES ET APPLIQEES
****
DEPARTEMENT DES SCIENCES TECHNOLOGIES
BP 206 Moundou, Tchad
Tel: (00235) 65291329
Email: udm@univ-mdou.org
****

جمهورية تشاد
****
وحدة – عمل – تقد
****
وزارة التعليم العالي
****
جامعة مو ندو
****
كلية العلوم الدقيقة والتطبيقية
****
قسم علوم التكنولوجيا

RAPPORT DE TRAVAUX PRATIQUE N° 01

THEME :

Application web de gestion des étudiants

Date : **Octobre 2026**

Conversion mobile réalisée par :

- ✓ **ALI Adoum Abakar**
- ✓ **CHAHAD Banane Moustapha**

Encadrant : **Prof. Arouna NDAM NJOYA**

Résumé

Ce rapport présente la conception et la réalisation de l'application web **« Gestion des étudiants »** de l'Université de Moundou, issue du TP1. L'application permet à un agent de la scolarité de **créer, consulter, modifier et supprimer** des fiches d'étudiants, de les **rechercher, filtrer, trier et paginer**, puis d'**éditer des cartes étudiantes** au format CR80 (85,6 × 54 mm) en PDF ou PNG, individuellement ou par lot.

Chaque carte porte un **QR code signé par HMAC-SHA256** : en le scannant, un contrôleur accède à une page publique qui confirme — ou infirme — l'authenticité de la carte sans exposer de données sensibles. L'application est écrite en **PHP 8 procédural** avec **PDO** (requêtes préparées natives), **MySQL/MariaDB** et **Bootstrap 5**, et s'exécute sur **XAMPP**. Le document décrit l'analyse des besoins, l'architecture, la base de données, chaque module, l'interface, les mesures de sécurité, la procédure d'installation, un plan de tests et les limites identifiées avec leurs pistes de correction.

Mots-clés : PHP 8, PDO, MySQL, Bootstrap 5, CRUD, carte étudiante, QR code, HMAC-SHA256, CSRF, Dompdf.

Table des matières

Résumé 2
1. Introduction 5
1.1 Contexte 5
1.2 Problématique 5
1.3 Objectifs 5
1.4 Périmètre 5
2. Analyse des besoins 6
2.1 Acteurs et cas d'utilisation 6
2.2 Besoins fonctionnels 6
2.3 Besoins non fonctionnels 7
3. Environnement et technologies 8
4. Architecture de l'application 9
4.1 Vue d'ensemble 9
4.2 Organisation des fichiers 9
4.3 Plan de navigation 10
5. Conception de la base de données 11
5.1 Modèle de données 11
5.2 Dictionnaire de données 11
5.3 Requêtes principales 12
6. Description détaillée des modules 13
6.1 Configuration centrale — config.php 13
6.2 Bibliothèque de fonctions — functions.php 13
6.3 Liste des étudiants — index.php 13
6.4 Formulaire et enregistrement — form.php et save.php 14
6.5 Suppression — delete.php 15
6.6 Recherche instantanée — api/search.php et app.js 15
6.7 Module des cartes étudiantes 16
6.8 Interface côté client 18
7. Interface utilisateur 19
7.1 Liste des étudiants 19
7.2 Formulaire avec erreurs de validation 20
7.3 Carte étudiante 21
7.4 Vérification d'une carte 22
7.5 Mode sombre 23
8. Sécurité 24
9. Installation et déploiement 25
9.1 Prérequis 25
9.2 Procédure 25
9.3 Export PNG 25

9.4 Avant une mise en production 25
10. Plan de tests 26
11. Limites identifiées et pistes d'amélioration 27
Perspectives 27
12. Conclusion 28
Annexe A — Script SQL de migration 29
Annexe B — Extraits de code significatifs 29
Annexe C — Glossaire 31

Si la table ci-dessus apparaît vide, faites un clic droit dessus dans Word puis « Mettre à jour les champs ».

1. Introduction

1.1 Contexte

La gestion des inscrits d'un établissement d'enseignement supérieur repose sur un fichier central : la liste des étudiants avec leur état civil, leur localisation et leur parcours (mention, cycle, parcours). À l'Université de Moundou, cette liste est stockée dans la table `etudiant` de la base `universite`, créée lors de la Leçon 2 / TP4. Le TP5 reprend cette base et la **refactorise** : le code devient plus sûr, mieux organisé et plus agréable à utiliser, puis s'enrichit d'un **module de cartes étudiantes**.

1.2 Problématique

Comment offrir à la scolarité un outil web simple pour tenir à jour la liste des étudiants, **sans exposer la base de données aux attaques courantes** (injection SQL, XSS, CSRF), tout en produisant des **cartes imprimables dont l'authenticité peut être vérifiée** par un tiers ?

1.3 Objectifs

- Implémenter un **CRUD complet** sur les étudiants avec validation côté serveur.
- Faciliter l'exploitation de la liste : **recherche instantanée, filtres, tri et pagination**.
- Générer des **cartes étudiantes CR80** (PDF, PNG, lot ZIP) avec photo ou initiales, QR code et code-barres décoratif.
- Garantir l'**authenticité** des cartes grâce à une signature HMAC-SHA256 vérifiable publiquement.
- Appliquer les bonnes pratiques de **sécurité** : requêtes préparées, jeton CSRF, échappement HTML, CSP.
- Proposer une **interface moderne, responsive et accessible** (mode sombre, lien d'évitement, libellés ARIA).

1.4 Périmètre

L'application est un outil **interne** déployé sur un serveur local XAMPP. Elle ne gère ni l'authentification des utilisateurs, ni les notes, ni les inscriptions pédagogiques : ces points sont signalés comme perspectives dans la section 11.

2. Analyse des besoins

2.1 Acteurs et cas d'utilisation

Deux acteurs interagissent avec l'application : l'**agent de la scolarité**, qui administre les fiches et produit les cartes, et le **contrôleur** (ou tout tiers) qui scanne le QR code d'une carte pour en vérifier l'authenticité. La figure 1 résume les cas d'utilisation.

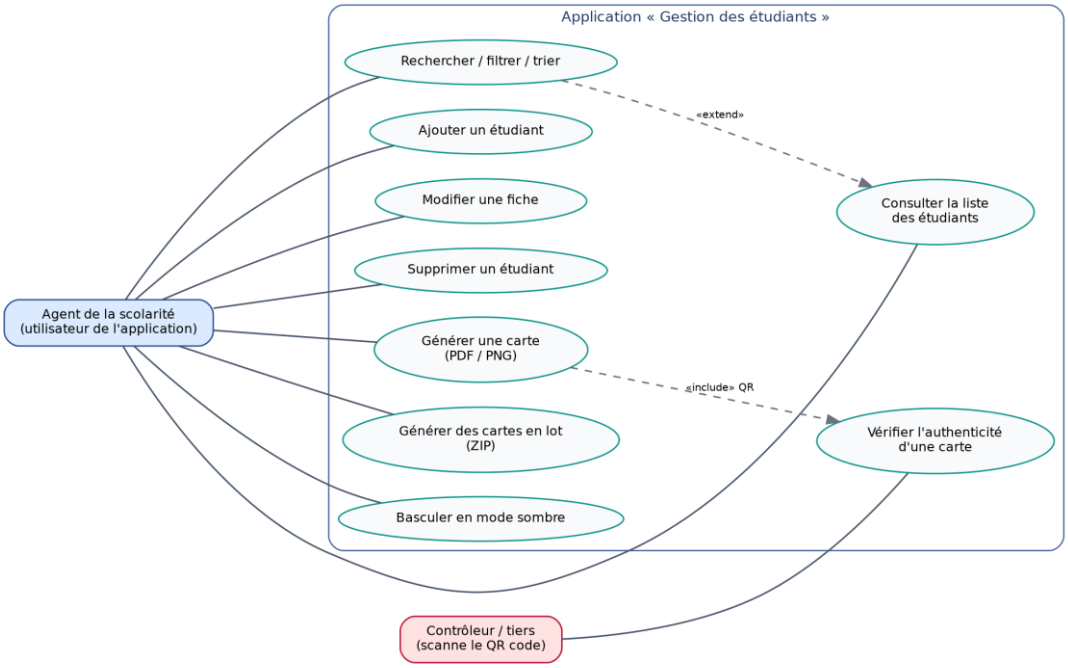


Figure 1 — *Diagramme des cas d'utilisation de l'application*

2.2 Besoins fonctionnels

Tableau 1 — *Besoins fonctionnels et pages qui les réalisent*

Réf.	Besoin	Réalisation
BF1	Afficher la liste des étudiants avec statistiques de synthèse	`index.php`
BF2	Rechercher par nom, prénom ou matricule (instantané et classique)	`index.php`, `api/search.php`, `app.js`
BF3	Filtrer par mention, cycle et département ; trier ; paginer (10 par page)	`index.php`, `functions.php`
BF4	Ajouter un étudiant avec contrôle des données saisies	`form.php`, `save.php`
BF5	Modifier une fiche (le matricule reste immuable)	`form.php`, `save.php`
BF6	Supprimer un étudiant avec confirmation	`delete.php`
BF7	Visualiser et télécharger la carte d'un étudiant (PDF, PNG), imprimer	`carte.php`, `carte_template.php`
BF8	Générer les cartes de plusieurs étudiants en une fois (ZIP)	`cartes_batch.php`
BF9	Journaliser chaque génération de carte	table `carte_etudiant`
BF10	Vérifier publiquement l'authenticité d'une carte via son QR code	`verifier.php`

2.3 Besoins non fonctionnels

Tableau 2 — *Exigences non fonctionnelles*

Domaine	Exigence	Mise en œuvre
Sécurité	Résister aux injections SQL, XSS et CSRF ; protéger les fichiers générés	PDO natif, `h()`, jeton CSRF, CSP, `.htaccess`
Ergonomie	Interface claire, retours visuels, aucune action destructrice par un simple lien	Toasts, spinner, confirmation, suppression en POST
Accessibilité	Navigation clavier, lecteurs d'écran, contrastes	Lien d'évitement, `aria-label`, focus visible
Performance	Rester fluide avec de nombreux étudiants	Pagination SQL (`LIMIT/OFFSET`), anti-rebond 250 ms
Portabilité	Fonctionner sur un poste standard	XAMPP (Apache, PHP ≥ 8.0, MySQL/MariaDB)
Maintenabilité	Code lisible et commenté, responsabilités séparées	Fichiers par fonction, constantes centralisées

3. Environnement et technologies

Le projet repose sur une pile classique **LAMP/XAMPP**, complétée par deux bibliothèques installées avec Composer pour la génération des cartes.

Tableau 3 — *Technologies utilisées*

Technologie	Version / source	Rôle dans le projet
PHP	≥ 8.0, mode `strict_types`	Langage serveur, style procédural
MySQL / MariaDB	Fourni par XAMPP	Base `universite` (tables `etudiant`, `carte_etudiant`)
PDO (pdo_mysql)	Extension PHP	Accès aux données, requêtes préparées natives
Apache	XAMPP, `.htaccess`	Serveur web, protection des fichiers, compression, cache
Bootstrap + Icons	5.3.3 / 1.11.3 (CDN jsDelivr)	Mise en page responsive, composants, icônes
JavaScript	Vanilla, `assets/js/app.js`	Thème, toasts, recherche live, sélection multiple
Dompdf	^3.0 (Composer)	Conversion HTML → PDF au format CR80, 300 ppp
Endroid QR Code	^5.0 (Composer)	Génération du QR code en `data:` URI
wkhtmltoimage / Imagick	Facultatifs	Export PNG haute résolution (Imagick exige Ghostscript)
ZipArchive (ext. zip)	Extension PHP	Archive des cartes générées en lot

4. Architecture de l’application

4.1 Vue d’ensemble

L’application suit une architecture en **quatre couches** : le navigateur (HTML, Bootstrap, JavaScript), le serveur PHP (pages, vues partagées, services transverses), les bibliothèques Composer et la couche de données (MySQL et dossier `cartes/`). Le style est **procédural** : aucune classe métier, mais une séparation nette entre configuration (`config.php`), fonctions réutilisables (`functions.php`), pages et gabarits.

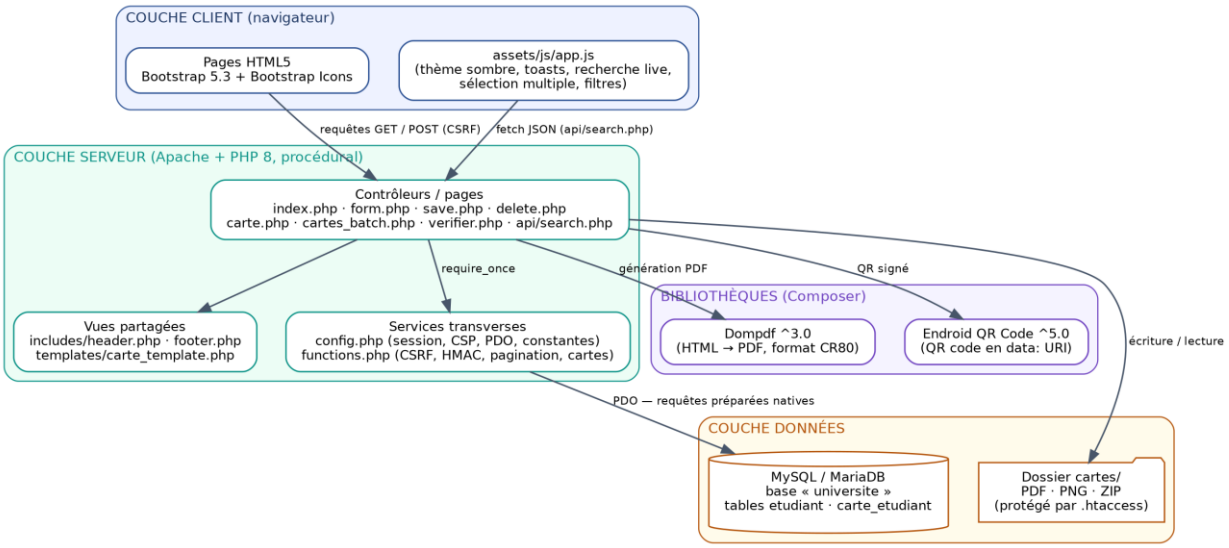


Figure 2 — *Architecture en couches de l’application*

4.2 Organisation des fichiers

```
tp5/
├── .htaccess                protection des fichiers sensibles, compression, cache
├── config.php               session, en-têtes de sécurité, connexion PDO, constantes, h()
├── functions.php            CSRF, HMAC, pagination, badges, QR, génération PDF/PNG
├── index.php                READ  : liste, recherche, filtres, tri, pagination
├── form.php                 formulaire partagé création / modification
├── save.php                 CREATE + UPDATE avec validation serveur
├── delete.php               DELETE (POST + CSRF + transaction)
├── carte.php                aperçu et téléchargement d’une carte
├── cartes_batch.php         génération de cartes en lot (ZIP)
├── verifier.php             vérification publique du QR code
├── composer.json            dépendances (Dompdf, Endroid QR Code)
├── schema_migration.sql     table carte_etudiant + index facultatifs
├── api/search.php           API JSON de la recherche live
├── includes/                header.php · footer.php
├── templates/               carte_template.php (recto / verso CR80)
├── assets/css/style.css     charte graphique, thème sombre
├── assets/js/app.js         comportements côté client
├── cartes/.htaccess         « Require all denied » (fichiers générés)
└── vendor/                  créé par « composer install »
```

Toutes les pages commencent par `require_once` de `config.php` puis de `functions.php`. Les pages d’affichage (`index.php`, `form.php`, `carte.php`, `verifier.php`) encadrent leur contenu par `includes/header.php` et `includes/footer.php`. Seules `carte.php` et `cartes_batch.php` chargent l’autoloader Composer, ce qui évite de payer le coût de Dompdf sur les autres pages.

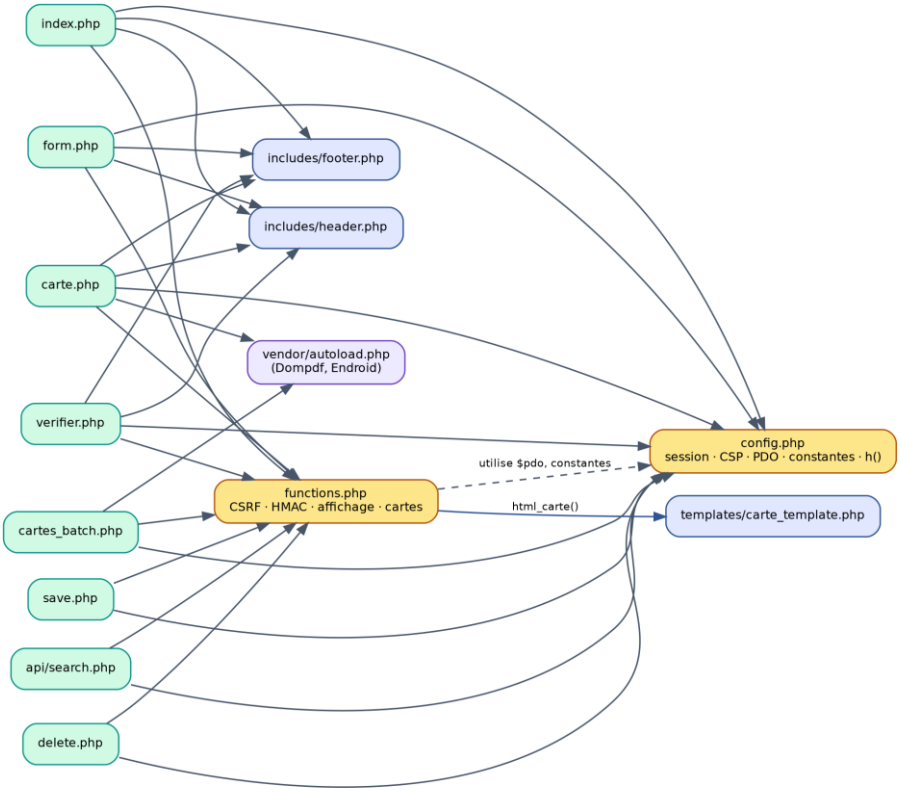


Figure 3 — *Dépendances entre les fichiers du projet*

4.3 Plan de navigation

Les écritures suivent le motif **Post / Redirect / Get** : `save.php`, `delete.php` et `cartes_batch.php` traitent la requête POST puis redirigent vers `index.php` (ou le formulaire en cas d'erreur), en déposant un message **flash** en session affiché sous forme de toast. Cela empêche la double soumission lors d'un rafraîchissement de page.

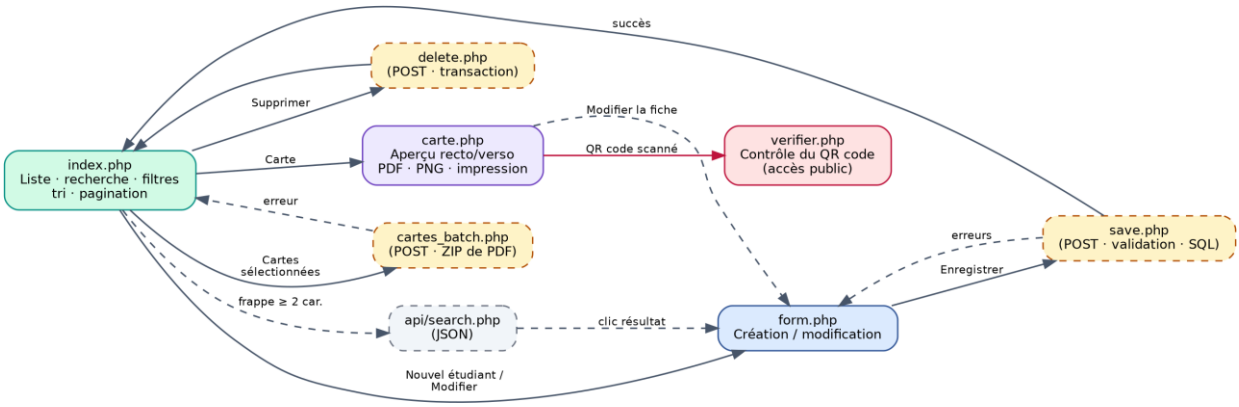


Figure 4 — *Plan de navigation entre les pages (traits pleins : liens ; pointillés : retours, erreurs, API)*

5. Conception de la base de données

5.1 Modèle de données

La base `universite` comporte deux tables. `etudiant` existait déjà (Leçon 2 / TP4) ; `carte_etudiant` est ajoutée par le script `schema_migration.sql`. Une étudiante ou un étudiant peut avoir **zéro à plusieurs cartes** générées au fil du temps ; une carte appartient à **un seul** étudiant. La clé étrangère est définie en `ON DELETE CASCADE` et `ON UPDATE CASCADE` : supprimer un étudiant supprime son historique de cartes.

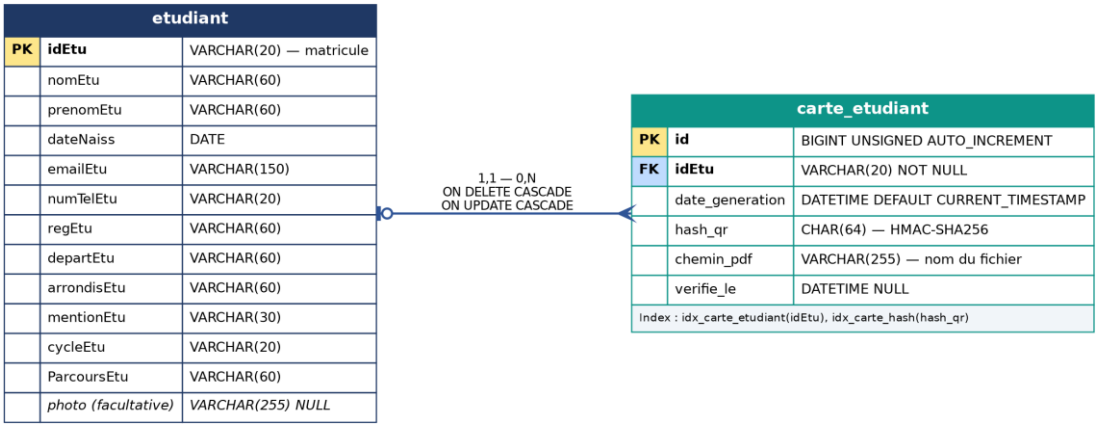


Figure 5 — *Modèle physique de données (types de `etudiant` déduits des longueurs validées par `save.php`)*

> **À vérifier :** La table `etudiant` n'est pas incluse dans l'archive du TP5 : ses types ont été déduits des longueurs maximales contrôlées dans `functions.php` (`LONGUEURS_MAX`) et des usages dans le code. Pour un schéma exact, exécutez `SHOW CREATE TABLE etudiant;` dans phpMyAdmin et reportez le résultat ici.

5.2 Dictionnaire de données

Tableau 4 — *Table etudiant — champs du formulaire (constante CHAMPS)*

Colonne	Libellé	Longueur max	Règle de validation
idEtu	Matricule	20	Obligatoire, clé primaire, non modifiable après création
nomEtu	Nom	60	Obligatoire
prenomEtu	Prénom	60	Obligatoire
dateNaiss	Date de naissance	DATE	Format AAAA-MM-JJ réel, non future, âge entre 15 et 80 ans
emailEtu	Email	150	`FILTER_VALIDATE_EMAIL` ; unicité signalée par SQLSTATE 23000
numTelEtu	Téléphone	20	Expression régulière `^\+?[0-9]{8,15}$`
regEtu	Région	60	Obligatoire, suggestions issues de la base
departEtu	Département	60	Obligatoire, suggestions issues de la base
arrondisEtu	Arrondissement	60	Obligatoire, suggestions issues de la base
mentionEtu	Mention	30	Obligatoire (ex. DI, IRS, Droit, Économie)
cycleEtu	Cycle	20	Liste : Licence, Master, Doctorat (valeur existante conservée)
ParcoursEtu	Parcours	60	Obligatoire — le « P » majuscule est celui de la colonne réelle

Tableau 5 — *Table carte_etudiant — journal des cartes générées*

Colonne	Type	Rôle
id	BIGINT UNSIGNED, AUTO_INCREMENT, PK	Identifiant interne du journal
idEtu	VARCHAR(20), NOT NULL, FK → etudiant	Matricule de l'étudiant concerné
date_generation	DATETIME, défaut CURRENT_TIMESTAMP	Horodatage de la génération
hash_qr	CHAR(64)	Signature HMAC-SHA256 du matricule (celle du QR code)
chemin_pdf	VARCHAR(255)	Nom du fichier dans `cartes/` (PDF, PNG ou lot)
verifie_le	DATETIME, NULL	Date de la première vérification du QR code

Le moteur est **InnoDB**, le jeu de caractères **utf8mb4** (collation `utf8mb4_unicode_ci`). Deux index accélèrent la consultation du journal : `idx_carte_etudiant (idEtu)` et `idx_carte_hash (hash_qr)`. Le script propose aussi, **en commentaire**, l'ajout d'une colonne `photo` à `etudiant` (utilisée par la carte lorsqu'elle est renseignée) et des index sur `nomEtu`, `mentionEtu`, `cycleEtu` et `departEtu`.

5.3 Requêtes principales

Tableau 6 — *Requêtes SQL clés de l'application*

Usage	Fichier	Requête (résumé)
Comptage pour la pagination	`index.php`	`SELECT COUNT(*) FROM etudiant [WHERE …]`
Page courante	`index.php`	`SELECT * FROM etudiant [WHERE …] ORDER BY <liste blanche> LIMIT :limite OFFSET :decalage`
Statistiques	`index.php`	`COUNT(*), SUM(cycleEtu = 'Licence'), SUM(cycleEtu = 'Master'), COUNT(DISTINCT departEtu)`
Création	`save.php`	`INSERT INTO etudiant (12 colonnes) VALUES (12 marqueurs nommés)`
Modification	`save.php`	`UPDATE etudiant SET … WHERE idEtu = :idEtu`
Suppression	`delete.php`	Transaction : `DELETE FROM carte_etudiant …` puis `DELETE FROM etudiant …`
Recherche live	`api/search.php`	`… WHERE nomEtu LIKE :q1 OR prenomEtu LIKE :q2 OR idEtu LIKE :q3 LIMIT 8`
Journal des cartes	`functions.php`, `carte.php`	`INSERT INTO carte_etudiant …` ; `SELECT … ORDER BY date_generation DESC LIMIT 10`
Vérification	`verifier.php`	`SELECT champs publics … WHERE idEtu = :id` ; `UPDATE carte_etudiant SET verifie_le = NOW() …`

6. Description détaillée des modules

6.1 Configuration centrale — config.php

Inclus par **toutes** les pages (y compris l'API), ce fichier remplit cinq rôles :

1. **Session sécurisée** : cookie `TP5SESSID`, `HttpOnly`, `SameSite=Lax`, drapeau `Secure` automatique en HTTPS ; fuseau horaire `Africa/Ndjamena`.
2. **En-têtes de sécurité** : `X-Content-Type-Options`, `X-Frame-Options`, `Referrer-Policy` et une **Content-Security-Policy** limitant les ressources au site, au CDN Bootstrap, aux images `data:` et au service de QR de secours.
3. **Connexion PDO** à `universite` avec `ERRMODE_EXCEPTION`, `FETCH_ASSOC` et surtout `ATTR_EMULATE_PREPARES = false` (requêtes préparées natives, nécessaires pour lier `LIMIT`/`OFFSET` en entier). Les erreurs de connexion sont journalisées sans révéler de détail à l'utilisateur (HTTP 503).
4. **Constantes globales** : version `2.0.0`, établissement, année académique `2025-2026`, clé secrète HMAC, dossier des cartes, 10 étudiants par page, liste des 12 champs.
5. **Fonction `h()`** : enveloppe de `htmlspecialchars` utilisée pour toute sortie HTML (protection XSS).

6.2 Bibliothèque de fonctions — functions.php

Ce fichier de près de 500 lignes regroupe les services transverses en six familles.

Tableau 7 — *Organisation de functions.php*

Famille	Fonctions principales	Rôle
A. Référentiels	`LIBELLES`, `TYPES`, `CYCLES`, `MENTIONS`, `TRI_COLONNES`, `LONGUEURS_MAX`, `REGEX_TEL`	Libellés, types de champs, listes blanches, limites
B. Session	`flash()`, `lire_flash()`, `memoriser_saisie()`, `saisie_precedente()`	Messages à usage unique, restitution de la saisie après erreur
C. Sécurité	`csrf_token()`, `csrf_field()`, `verifier_csrf()`, `rediriger()`, `token_carte()`, `token_valide()`, `url_base()`	Anti-CSRF, signature et vérification HMAC
D. Affichage	`initiales()`, `couleur_depuis_chaine()`, `badge_mention()`, `badge_cycle()`, `avatar()`, `colonne_triable()`, `url_avec()`, `lien_tri()`, `pagination_html()`	Avatars colorés, badges, tri serveur, pagination
E. Cartes	`qr_src()`, `logo_src()`, `photo_src()`, `url_verification()`, `date_validite()`, `code_barres_html()`, `html_carte()`	Composition d'une carte
F. Fichiers	`preparer_dossier_cartes()`, `dimensions_cr80()`, `pdf_carte()`, `png_carte()`, `enregistrer_carte()`	Production, stockage et journalisation

6.3 Liste des étudiants — index.php

La page lit les paramètres d'URL (`q`, `mention`, `cycle`, `depart`, `tri`, `ordre`, `page`), les nettoie, puis construit dynamiquement une clause `WHERE`. **Aucune valeur utilisateur n'est concaténée** dans le SQL, hormis le nom de colonne du `ORDER BY`, qui provient d'une **liste blanche** (`colonne_triable()`).

```
$conditions = []; $parametres = [];
if ($q !== '') {
    $conditions[] = '(nomEtu LIKE :q1 OR prenomEtu LIKE :q2 OR idEtu LIKE :q3)';
    $parametres[':q1'] = $parametres[':q2'] = $parametres[':q3'] = '%' . $q . '%';
}
if ($mention !== '') { $conditions[] = 'mentionEtu = :mention'; $parametres[':mention'] = $mention; }
// ... idem pour cycle et département
$clause = $conditions ? ' WHERE ' . implode(' AND ', $conditions) : '';

$requete = $pdo->prepare("SELECT * FROM etudiant" . $clause .
    " ORDER BY $tri $ordre LIMIT :limite OFFSET :decalage");
$requete->bindValue(':limite',   PAR_PAGE, PDO::PARAM_INT);
$requete->bindValue(':decalage', $offset,  PDO::PARAM_INT);
```

Le marqueur de recherche est répété (`:q1`, `:q2`, `:q3`) car les requêtes préparées **natives** n'autorisent pas la réutilisation d'un même marqueur nommé. La page affiche ensuite quatre cartes de statistiques, la barre de recherche et de filtres (les listes déroulantes sont alimentées par des `SELECT DISTINCT`), le tableau triable avec avatars et badges colorés, et la pagination qui conserve tous les filtres grâce à `url_avec()`.

6.4 Formulaire et enregistrement — form.php et save.php

`form.php` sert à la fois à la **création** (sans paramètre) et à la **modification** (`?id=XXX`). En modification, le matricule est en lecture seule ; si l'étudiant n'existe pas, un message est affiché et l'utilisateur est renvoyé à la liste. Les champs région, département, arrondissement, mention et parcours proposent des **suggestions** (`<datalist>`) tirées de la base ; le cycle est une liste déroulante. Après une erreur, la saisie est restituée depuis la session.

`save.php` ne fait **jamais confiance au navigateur** : il rejoue côté serveur toutes les validations, cumule les erreurs, puis écrit en base.

Tableau 8 — *Contrôles effectués par save.php, dans l'ordre*

N°	Contrôle	Réponse en cas d'échec
1	Méthode HTTP = POST	HTTP 405 « Méthode non autorisée »
2	Jeton CSRF valide (`verifier_csrf()`)	HTTP 419, message flash, retour à la liste
3	Présence de chacun des 12 champs et longueur maximale	Message d'erreur par champ
4	Format de l'email (`FILTER_VALIDATE_EMAIL`)	Message d'erreur
5	Format du téléphone (8 à 15 chiffres, « + » facultatif)	Message d'erreur
6	Date de naissance réelle, non future, âge entre 15 et 80 ans	Message d'erreur
7	En modification : l'étudiant existe	Message d'erreur
8	Écriture `INSERT` / `UPDATE` ; doublon (SQLSTATE 23000)	Message « matricule ou email déjà utilisé »

En modification, la requête `UPDATE` est construite à partir de la constante `CHAMPS` privée de `idEtu`, qui sert uniquement de critère `WHERE` : le matricule ne peut donc pas être altéré, même par une requête forgée.

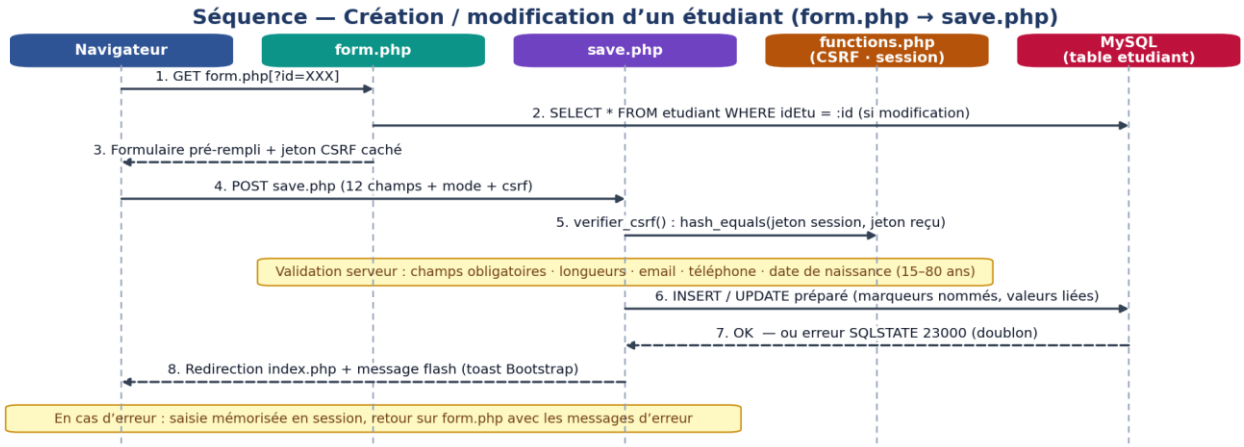


Figure 6 — *Diagramme de séquence de la création / modification d'un étudiant*

6.5 Suppression — delete.php

Une suppression ne doit **jamais** être déclenchée par un lien GET : un moteur de préchargement ou une image distante pourrait la provoquer. Le bouton de la liste est donc un bouton de soumission lié à un formulaire POST caché contenant le jeton CSRF, précédé d'une boîte de confirmation JavaScript. Côté serveur, `delete.php` contrôle la méthode et le jeton, vérifie que l'étudiant existe, puis exécute dans une **transaction** la suppression de ses cartes (`carte_etudiant`) et de sa fiche (`etudiant`). En cas d'erreur, un `rollBack()` annule l'ensemble ; la contrainte SQLSTATE 23000 produit un message explicite si d'autres enregistrements référencent l'étudiant.

6.6 Recherche instantanée — api/search.php et app.js

Dès que l'utilisateur saisit **au moins deux caractères** dans la barre de recherche, `app.js` attend **250 ms** après la dernière frappe (anti-rebond) puis interroge `api/search.php?q=…` avec `fetch`. L'API tronque le terme à 60 caractères, exécute une requête préparée limitée à **8 résultats** et renvoie un JSON `{"resultats": [...]}` ; la liste de suggestions s'affiche sous le champ, chaque entrée menant au formulaire de modification de l'étudiant. Un clic à l'extérieur ferme la liste.

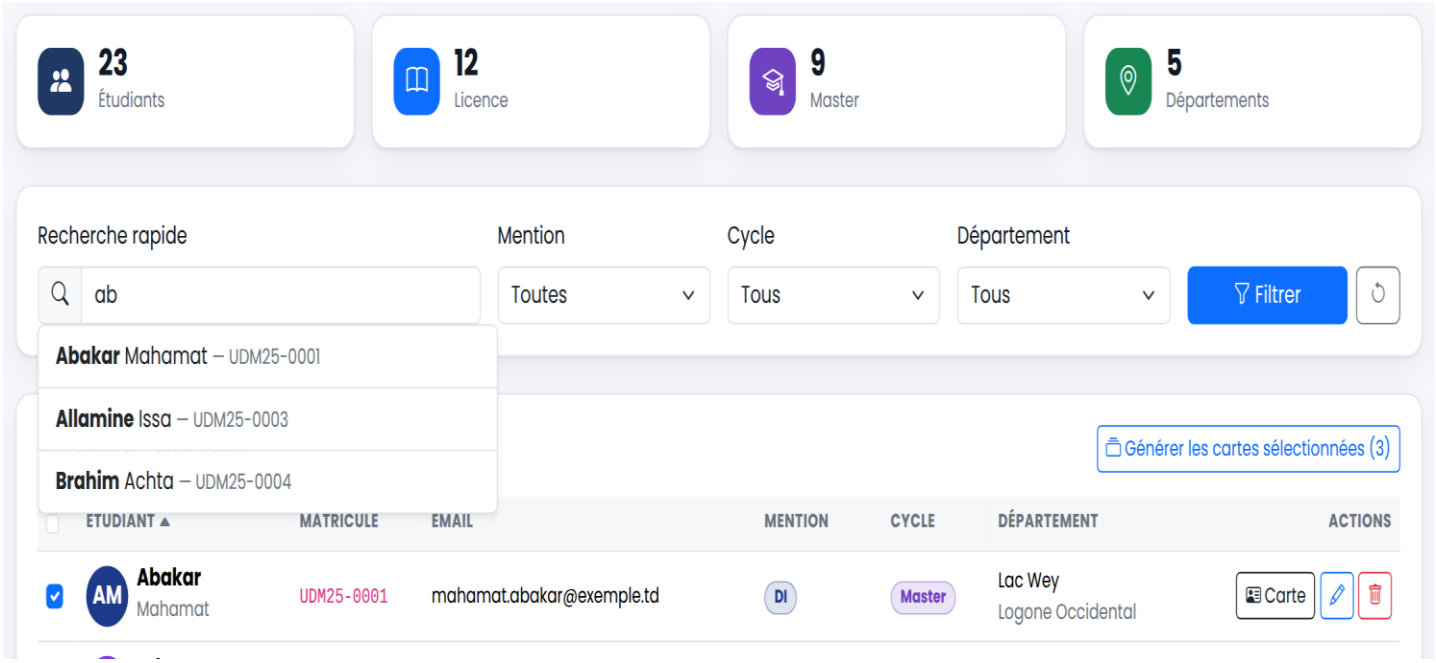


Figure 7 — *Recherche instantanée : suggestions pour « ab »*

6.7 Module des cartes étudiantes

Le gabarit et le format CR80

`templates/carte_template.php` produit le **recto** : bandeau coloré avec logo (ou initiale de secours), nom de l'établissement, année académique et QR code ; corps avec photo (ou initiales sur fond coloré), matricule, nom, mention, cycle, parcours, département, région et date de naissance ; pied avec date de validité (31/08/2026), code-barres décoratif et mention légale. Le **verso** (facultatif) affiche les conditions d'utilisation, les zones de signature et de cachet. Les dimensions sont exprimées en **millimètres** (85,6 × 54 mm, marge de sécurité de 3 mm) et la couleur de la carte est dérivée du matricule, ce qui la rend stable d'une génération à l'autre. En génération par lot, un **filigrane « COPIE »** est ajouté.



Figure 8 — *Carte étudiante CR80 — recto (avec QR code) et verso (rendu d'illustration)*

Aperçu, PDF, PNG et journal — carte.php

Tableau 9 — *Paramètres de carte.php*

URL	Résultat
`carte.php?id=XXX`	Aperçu HTML du recto, boutons de téléchargement, historique des 10 dernières générations
`carte.php?id=XXX&verso=1`	Aperçu avec le verso
`carte.php?id=XXX&format=pdf`	Téléchargement du PDF (CR80, 300 ppp, Dompdf)
`carte.php?id=XXX&format=png`	Téléchargement du PNG (wkhtmltoimage, sinon Imagick + Ghostscript)

À chaque téléchargement, `enregistrer_carte()` écrit le fichier dans `cartes/` sous le nom `carte_<matricule>_<AAAAMMJJ-HHMMSS>.<ext>` et insère une ligne dans `carte_etudiant` avec la signature du QR. Si la génération échoue (extension absente, Dompdf non installé), l'utilisateur reçoit un message d'avertissement et l'aperçu HTML reste consultable et imprimable.

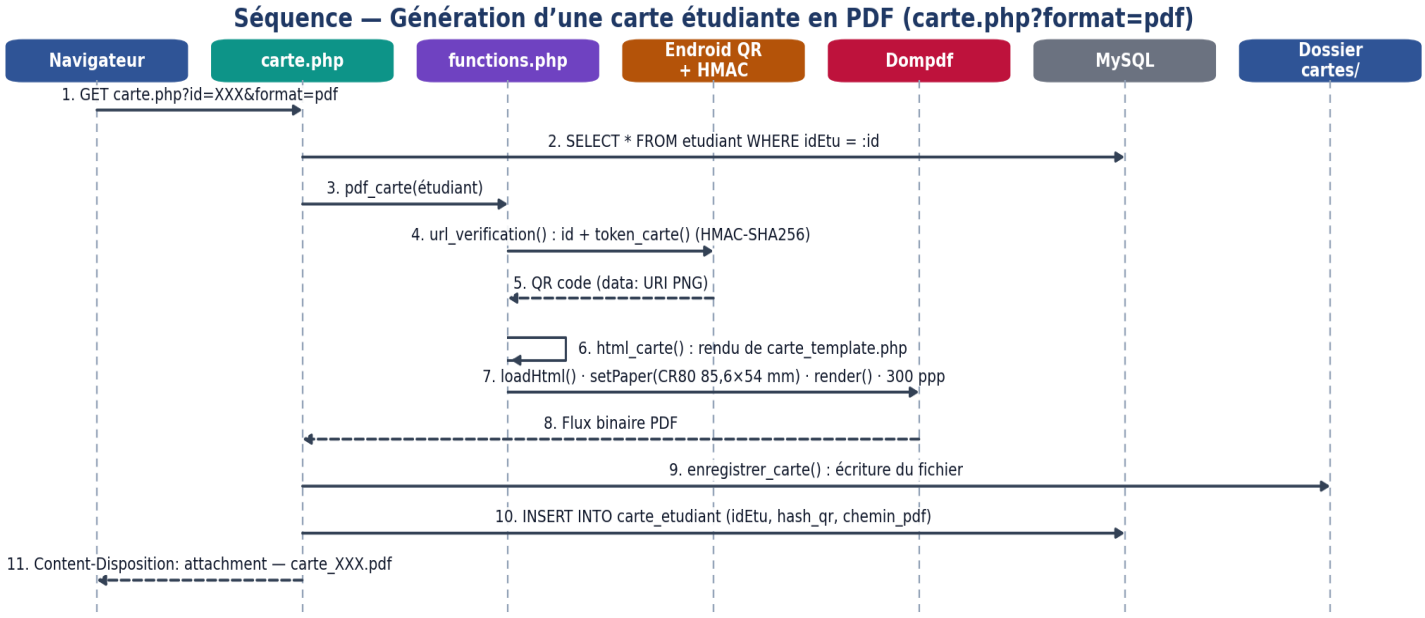


Figure 9 — *Diagramme de séquence de la génération d'une carte PDF*

Génération par lot — cartes_batch.php

L'utilisateur coche des lignes dans `index.php` (case « tout sélectionner » disponible, compteur dynamique sur le bouton) et envoie le formulaire en POST. `cartes_batch.php` contrôle la méthode et le jeton CSRF, nettoie la sélection (limitée à **200 cartes**), charge les fiches avec une requête `IN (?, ?, …)` aux marqueurs générés dynamiquement, génère un PDF par étudiant avec le filigrane « COPIE », les journalise, puis les regroupe avec un fichier `liste.txt` dans une **archive ZIP** envoyée au navigateur.

Vérification publique — verifier.php

Le QR code encode l'URL `verifier.php?id=<matricule>&token=<HMAC>`. Le jeton est calculé par `hash_hmac('sha256', $idEtu, CLE_SECRETE)` ; sans la clé, il est **impossible à forger**, et toute modification du matricule invalide la signature. La comparaison utilise `hash_equals()` (temps constant). La page publique n'affiche que des informations non sensibles — **ni email ni téléphone** — et enregistre la date de première vérification dans `carte_etudiant.verifie_le`.

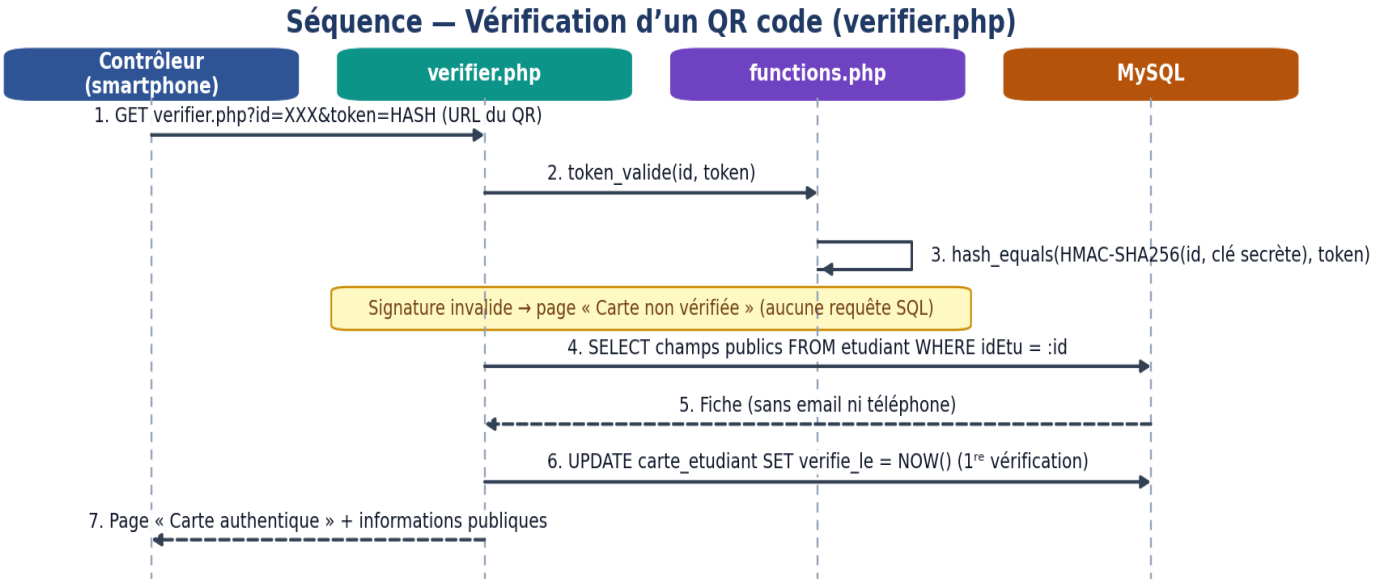


Figure 10 — *Diagramme de séquence de la vérification d'un QR code*

6.8 Interface côté client

Tableau 10 — *Fonctions de app.js*

N°	Comportement	Détail
1	Mode sombre persistant	Bascule `data-bs-theme`, mémorisée dans `localStorage` (clé `tp5-theme`) et appliquée avant le premier rendu pour éviter tout clignotement
2	Toasts	Les messages flash injectés par `header.php` sont affichés via `bootstrap.Toast` (disparition après 6 s)
3	Loader de soumission	Spinner et désactivation du bouton pour éviter les doubles clics (`form[data-chargement]`)
4	Recherche live	`fetch` vers `api/search.php`, anti-rebond 250 ms, minimum 2 caractères
5	Sélection multiple	Case « tout cocher » et compteur du bouton « Générer les cartes sélectionnées (n) »
6	Filtres automatiques	Soumission du formulaire dès le changement d'une liste déroulante ; impression de l'aperçu de carte

`style.css` définit la charte « dashboard universitaire » : jetons de design (bleu académique `#1F3864`, rayons, ombres), thème sombre, cartes de statistiques, tableau, badges colorés par `color-mix`, avatars, lien d'évitement et focus visible conforme aux contrastes AA.

7. Interface utilisateur

> **Remarque :** Les captures ci-dessous sont des rendus de l'interface générés à partir du code HTML et de la feuille de style du projet, avec des **données d'exemple fictives** (noms et matricules inventés). Elles illustrent la mise en page ; remplacez-les par vos captures réelles (touche Impr. écran) si votre enseignant l'exige.

7.1 Liste des étudiants

La page d'accueil réunit les statistiques, la recherche, les trois filtres, le tableau triable (l'en-tête actif affiche une flèche), la sélection multiple pour les cartes en lot et la pagination.

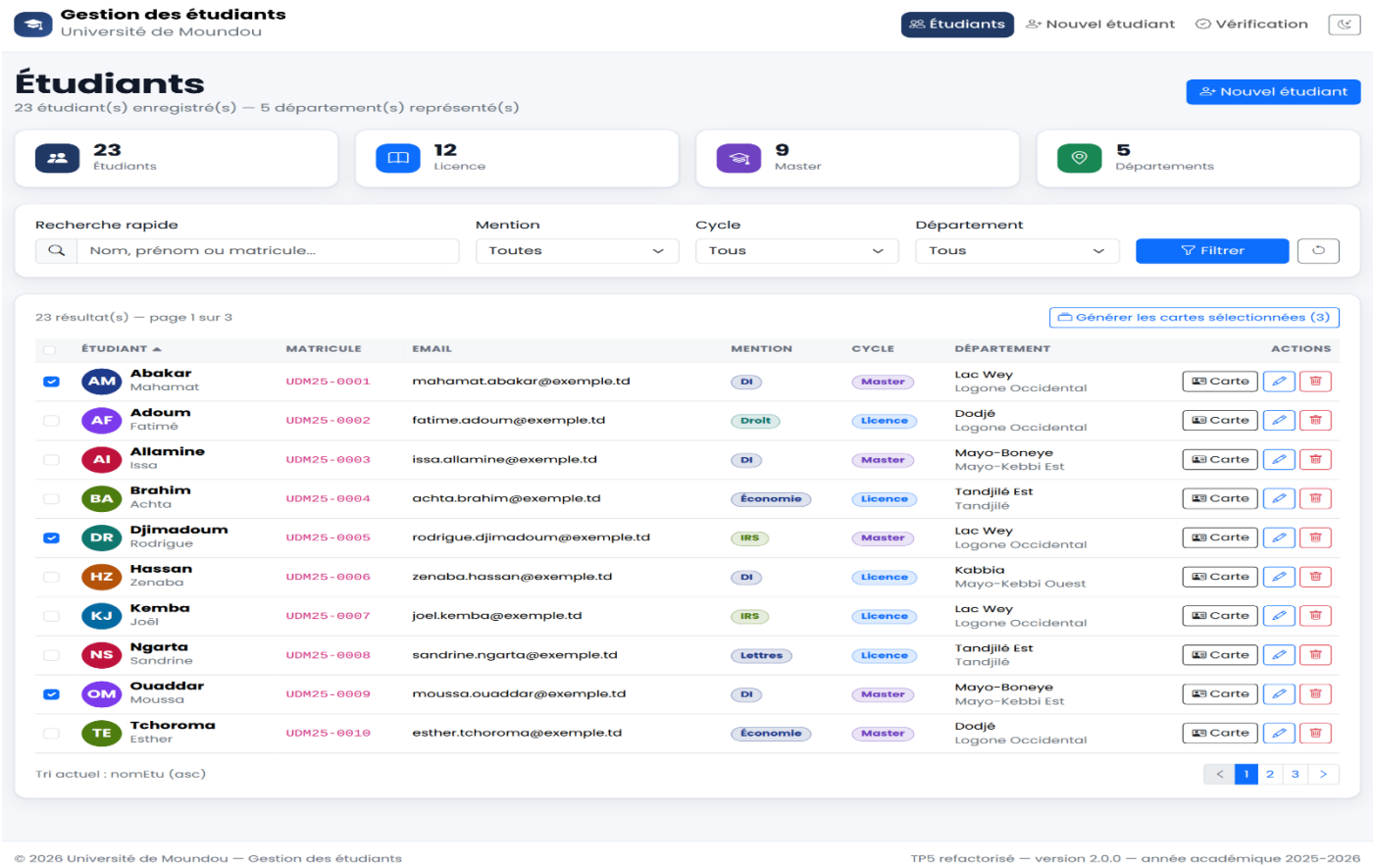


Figure 11 — *Page d'accueil — liste des étudiants (page 1 sur 3, trois lignes cochées)*

7.2 Formulaire avec erreurs de validation

En cas de saisie invalide, `save.php` renvoie vers le formulaire en conservant les valeurs saisies ; les erreurs sont affichées sous forme de toasts rouges (ici, un email et un téléphone incorrects).

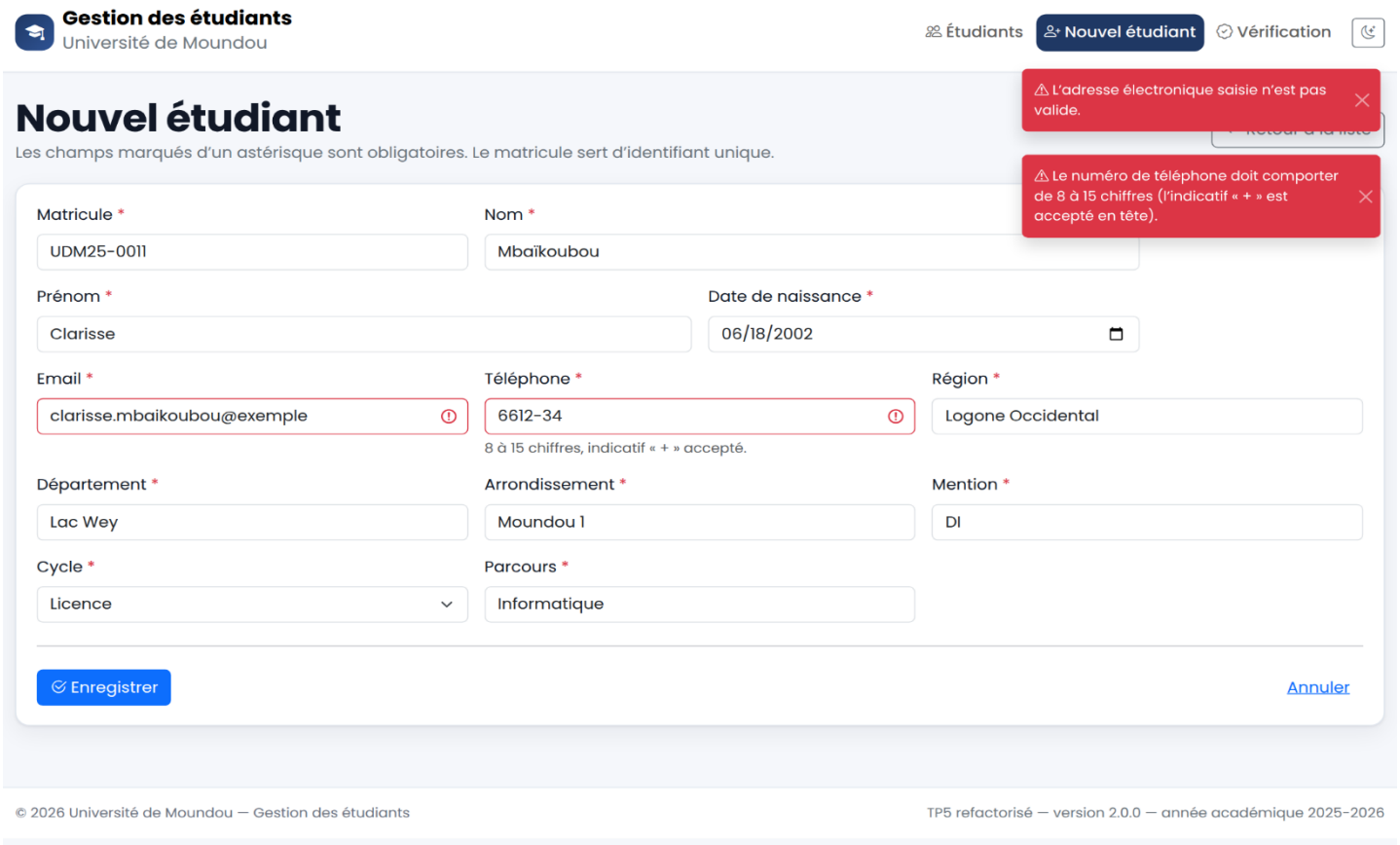


Figure 12 — *Formulaire « Nouvel étudiant » après un échec de validation*

7.3 Carte étudiante

La page de la carte propose le téléchargement en PDF ou PNG, l'impression, l'affichage du verso et le journal des générations.

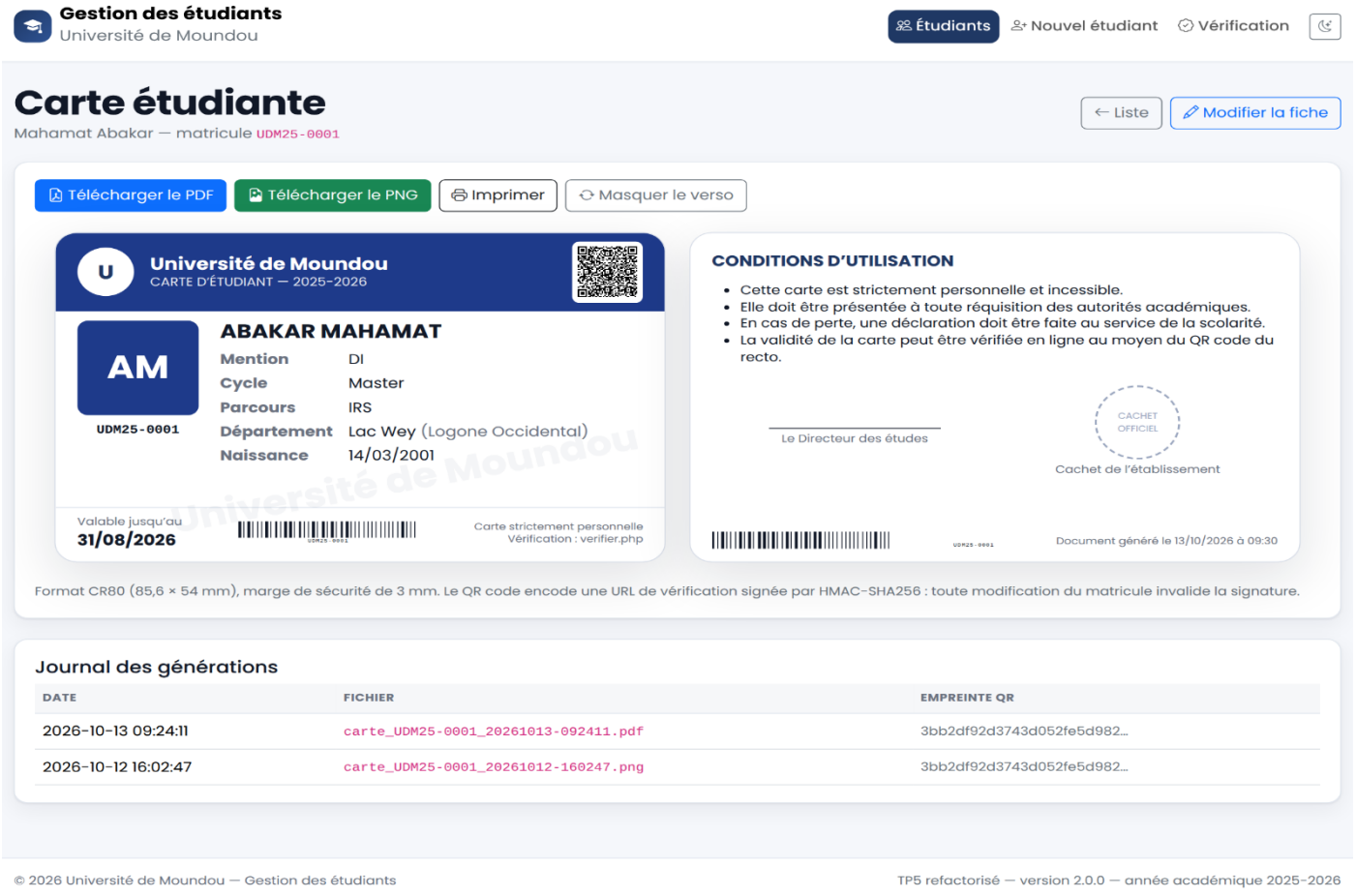


Figure 13 — *Page carte.php — aperçu recto/verso et journal des générations*

> **Attention :** Le gabarit `carte_template.php` utilise des classes (`carte-recto`, `bandeau`, `corps`, …) dont la feuille de style n'est pas fournie dans l'archive. La mise en forme ci-dessus est donc une reconstitution fidèle à la structure du gabarit ; voir la limite n° 1 en section 11.

7.4 Vérification d’une carte

Selon la validité de la signature, la page publique affiche une confirmation avec les informations non sensibles de l’étudiant, ou un message de rejet.

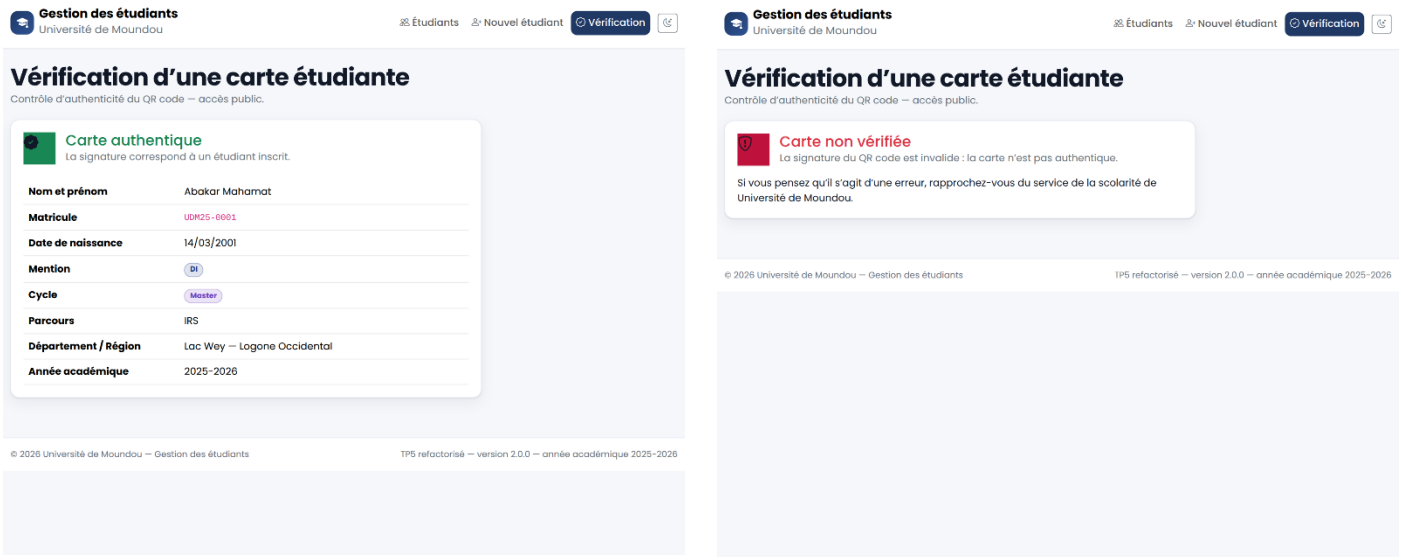


Figure 14 — *Vérification d’une carte — à gauche : « Carte authentique » ; à droite : signature invalide*

7.5 Mode sombre

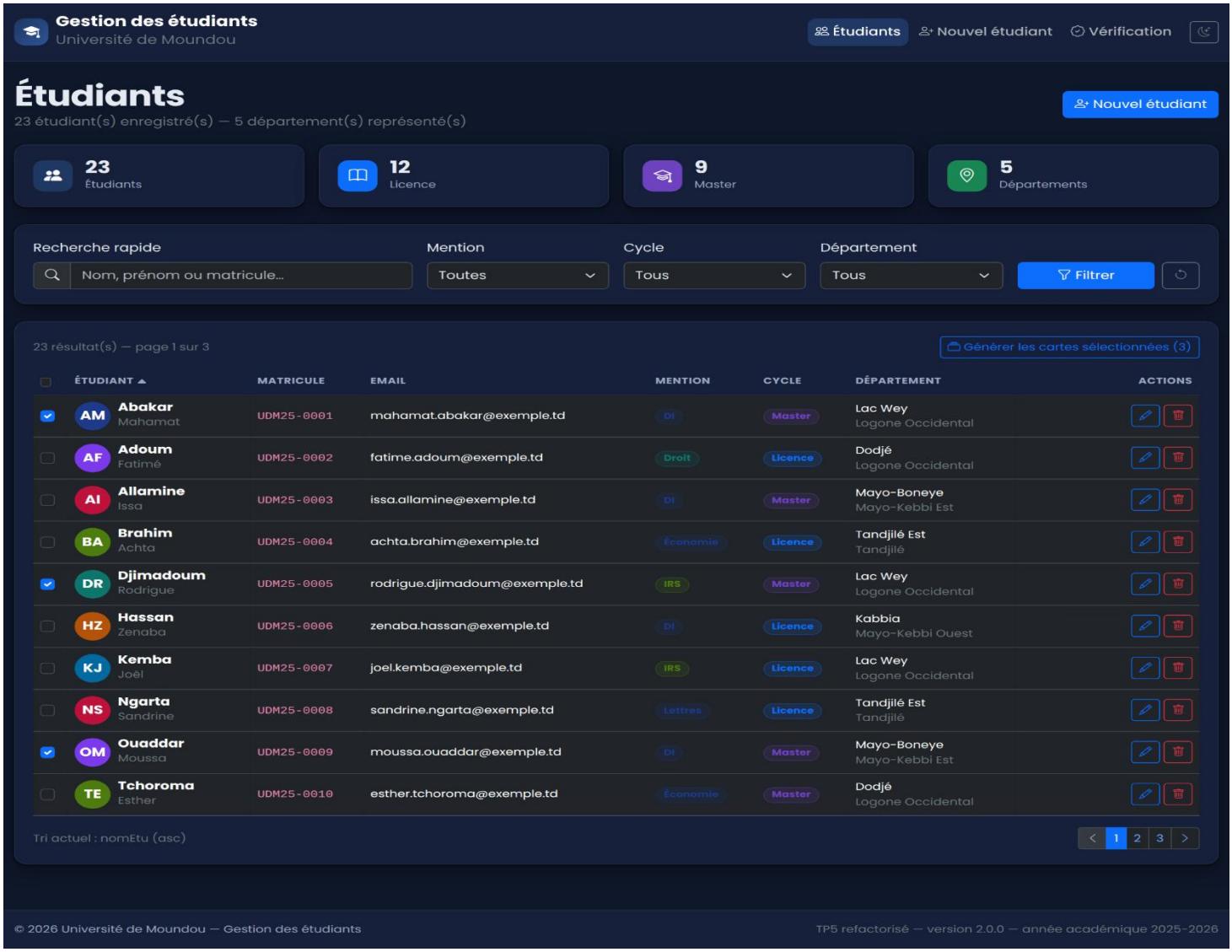


Figure 15 — *Liste des étudiants en mode sombre (bouton lune/soleil de la barre de navigation)*

8. Sécurité

La sécurité est traitée en **défense en profondeur** : chaque requête traverse plusieurs barrières indépendantes, de l'en-tête HTTP jusqu'à l'affichage.

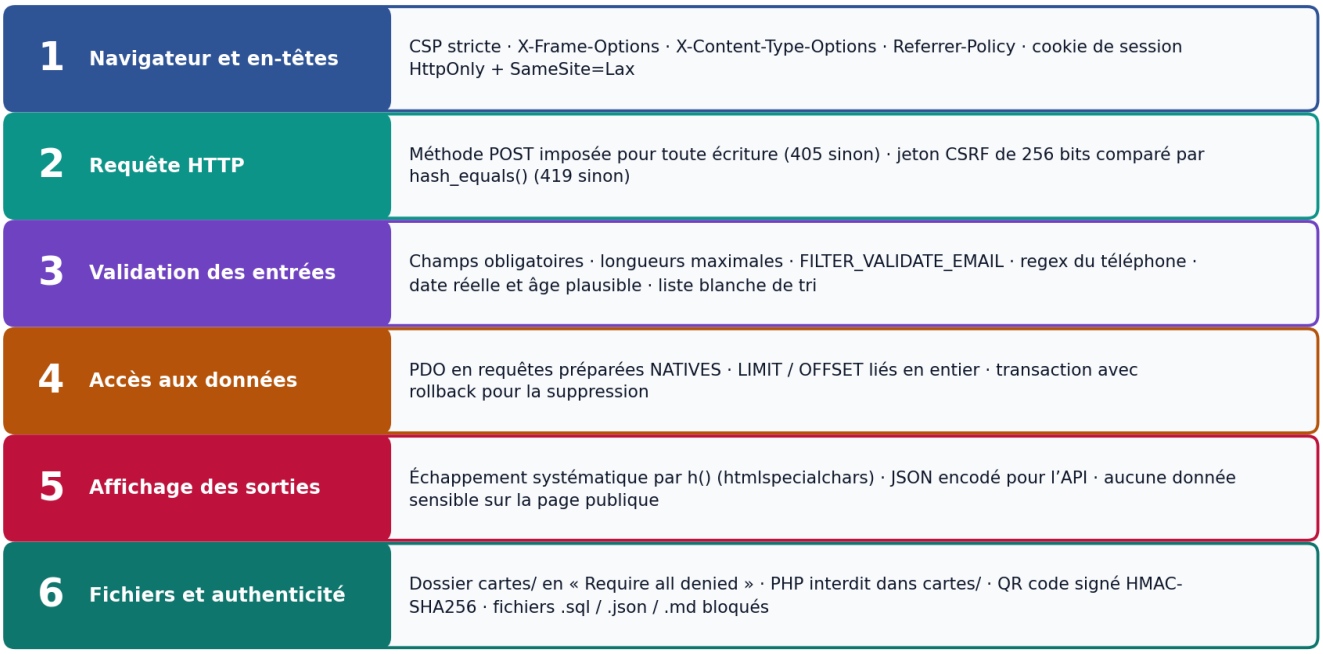


Figure 16 — *Les six couches de protection de l'application*

Tableau 11 — *Menaces et contre-mesures*

Menace	Contre-mesure	Où ?
Injection SQL	Requêtes préparées natives ; `ORDER BY` par liste blanche ; `LIMIT`/`OFFSET` liés en entier	`config.php`, `index.php`, `save.php`
XSS (injection de script)	Échappement par `h()` ; CSP stricte ; API JSON sans HTML	`config.php`, toutes les vues
CSRF	Jeton de 256 bits en session, champ caché, `hash_equals()` ; POST obligatoire pour toute écriture ; cookie `SameSite=Lax`	`functions.php`, `save.php`, `delete.php`, `cartes_batch.php`
Falsification de carte	QR signé HMAC-SHA256 ; vérification en temps constant	`functions.php`, `verifier.php`
Accès direct aux fichiers générés	`cartes/.htaccess` (« Require all denied ») ; exécution de PHP interdite dans `cartes/`	`.htaccess`, `cartes/.htaccess`
Fuite de fichiers sensibles	Blocage de `.env`, `.json`, `.lock`, `.sql`, `.md`, `.log` ; `Options -Indexes`	`.htaccess`
Fuite d'informations techniques	Erreurs PDO journalisées (`error_log`) sans détail à l'écran	`config.php`, `save.php`
Clickjacking / sniffing	`X-Frame-Options: SAMEORIGIN`, `X-Content-Type-Options: nosniff`	`config.php`
Exposition de données personnelles	Page publique limitée aux champs non sensibles (pas d'email ni de téléphone)	`verifier.php`

9. Installation et déploiement

9.1 Prérequis

- **XAMPP** : Apache, PHP 8.0 ou supérieur, MySQL / MariaDB.
- **Composer** pour installer Dompdf et Endroid QR Code.
- Extensions PHP : `pdo_mysql`, `mbstring`, `zip` (lot) ; `imagick` facultative (PNG).
- Base `universite` contenant déjà la table `etudiant` (Leçon 2 / TP4).

9.2 Procédure

1. Copier le dossier `tp5` dans `C:\xampp\htdocs`.
2. Dans un terminal : `cd C:\xampp\htdocs\tp5` puis `composer install`.
3. Démarrer **Apache** et **MySQL** depuis le panneau de contrôle XAMPP.
4. Dans phpMyAdmin, ouvrir la base `universite`, onglet **SQL**, coller le contenu de `schema_migration.sql` et exécuter.
5. Vérifier que le dossier `cartes/` est accessible en écriture par Apache.
6. (Facultatif) Déposer un logo dans `assets/img/logo.png` et des photos dans `assets/img/photos/`, puis ajouter la colonne `photo` si besoin.
7. Ouvrir **http://localhost/tp5/** dans le navigateur.

9.3 Export PNG

Deux moteurs sont essayés dans l'ordre : **wkhtmltoimage** (recommandé ; ajouter `wkhtmltopdf\bin` au `PATH`), puis **Imagick** appliqué au PDF (extension `imagick` et Ghostscript requis). À défaut, l'export PDF reste disponible.

9.4 Avant une mise en production

- Remplacer la clé `CLE_SECRETE` par une valeur aléatoire longue, stockée **hors du code** (fichier `.env` non versionné).
- Créer un compte MySQL dédié avec des droits limités, à la place de `root` sans mot de passe.
- Servir l'application en **HTTPS** (le cookie de session passe alors automatiquement en `Secure`).
- Ajouter une **authentification** (voir section 11).

10. Plan de tests

Le tableau suivant propose des scénarios de validation couvrant les fonctions et les protections de l'application. La colonne « Résultat obtenu » est à compléter lors de la recette.

Tableau 12 — *Scénarios de test*

N°	Scénario	Résultat attendu	Résultat obtenu
T1	Ouvrir `index.php` avec des données	Liste paginée (10 lignes), statistiques exactes	
T2	Rechercher « ab » dans la recherche live	Au plus 8 suggestions, aucune requête sous 2 caractères	
T3	Filtrer Mention = DI puis cycle = Master	Seules les lignes correspondantes ; pagination conserve les filtres	
T4	Cliquer sur l'en-tête « Email » deux fois	Tri ascendant puis descendant, flèche affichée	
T5	Créer un étudiant valide	Toast vert, étudiant visible dans la liste	
T6	Créer avec un email invalide	Toast rouge, saisie conservée, aucune insertion	
T7	Créer avec un matricule déjà existant	Message de doublon, aucune insertion	
T8	Saisir une date de naissance dans le futur	Message d'erreur	
T9	Modifier une fiche	Mise à jour ; le matricule est en lecture seule	
T10	Supprimer un étudiant (confirmer)	Fiche et cartes supprimées, toast de succès	
T11	Envoyer un POST sans jeton CSRF	HTTP 419, aucune écriture	
T12	Ouvrir `delete.php` par un lien GET	HTTP 405, aucune suppression	
T13	Télécharger le PDF d'une carte	PDF 85,6 × 54 mm, ligne ajoutée au journal	
T14	Générer 3 cartes en lot	Archive ZIP avec 3 PDF et `liste.txt`	
T15	Scanner le QR d'une carte valide	Page « Carte authentique »	
T16	Modifier un caractère du jeton dans l'URL	Page « Carte non vérifiée »	
T17	Accéder directement à `/cartes/fichier.pdf`	Accès refusé (403)	
T18	Activer le mode sombre puis recharger	Thème conservé sans clignotement	

11. Limites identifiées et pistes d'amélioration

La relecture complète du code a mis en évidence plusieurs points à corriger ou à renforcer. Ils sont classés ci-dessous par priorité décroissante.

Tableau 13 — *Limites constatées*

N°	Limite constatée	Piste de correction
1	**Style de la carte manquant.** Les classes de `carte_template.php` (`carte-recto`, `bandeau`, `corps`, `pied`, `carte-verso`…) ne sont définies dans aucune feuille de style de l'archive ; `style.css` ne contient que `.carte-cr80`, que le gabarit n'utilise pas. L'aperçu et le PDF s'affichent donc sans mise en forme.	Ajouter un bloc `<style>` dans le gabarit (aussi pris en compte par Dompdf), en millimètres, avec les classes utilisées.
2	**PDF de lot invalide.** `cartes_batch.php` assemble le « PDF multipages » en concaténant des flux PDF avec `implode()` ; ce fichier n'est généralement pas lisible, et n'est de toute façon pas renvoyé (seul le ZIP l'est). Un premier rendu Dompdf est aussi calculé pour rien.	Construire un seul HTML contenant toutes les cartes avec un saut de page entre chacune, puis un seul `render()`.
3	**Recherche live et HTML.** `app.js` insère `nomEtu`, `prenomEtu` et `idEtu` dans `innerHTML` sans échappement : un nom contenant du HTML serait interprété. La CSP (`script-src 'self'`) limite l'exécution de scripts, mais l'injection de balises reste possible.	Créer les éléments avec `textContent` ou échapper les valeurs avant insertion.
4	**Aucune authentification.** Toute personne atteignant le serveur peut créer, modifier, supprimer et générer des cartes.	Ajouter une page de connexion, des sessions authentifiées et des rôles (agent, administrateur).
5	**Secrets dans le code.** La clé HMAC est écrite dans `config.php` ; MySQL est accédé avec `root` sans mot de passe.	Variables d'environnement ou `.env` hors dépôt ; compte MySQL dédié.
6	**QR sans expiration ni révocation.** La signature ne dépend que du matricule : une carte perdue ou périmée reste « authentique ».	Inclure l'année académique ou un identifiant de carte dans la signature, vérifier le statut en base.
7	**Photo et logo absents.** `assets/img/` n'existe pas dans l'archive ; la carte utilise des initiales et un logo de secours. La colonne `photo` est facultative.	Fournir le logo officiel et prévoir un envoi de photo sécurisé (type, taille, renommage).
8	**Divers.** Les caractères `%` et `_` saisis dans la recherche ne sont pas échappés ; la CSP autorise `'unsafe-inline'` pour les styles ; Bootstrap est chargé depuis un CDN sans contrôle d'intégrité (SRI) ; la colonne `chemin_pdf` stocke aussi des PNG ; aucun test automatisé.	Échapper les jokers `LIKE`, ajouter SRI ou héberger Bootstrap localement, renommer la colonne, écrire des tests PHPUnit.

Perspectives

- Export de la liste en **CSV / Excel** et **import** en masse.
- Gestion des **photos** d'identité avec recadrage.
- **Journal d'audit** des modifications et suppressions.
- Passage progressif à une architecture **MVC** (ou à un framework comme Laravel) pour faciliter l'évolution.

12. Conclusion

Le TP5 refactorisé transforme un CRUD élémentaire en une application complète de gestion des étudiants : saisie contrôlée, exploitation confortable de la liste, cartes étudiantes personnalisées et vérifiables, le tout dans une interface moderne et accessible. L'accent mis sur la sécurité — requêtes préparées natives, jeton CSRF, échappement systématique, CSP, signature HMAC — en fait une base saine pour la suite.

La revue du code révèle toutefois des points à finaliser avant tout usage réel, principalement la **feuille de style de la carte**, la **génération du PDF de lot**, l'**échappement dans la recherche live** et l'**authentification**. Leur correction est de taille raisonnable et ferait passer l'application du statut de travail pratique abouti à celui d'outil exploitable par la scolarité.

Ce projet a permis de mettre en pratique le développement web côté serveur avec PHP et MySQL, la conception d'une base relationnelle, la sécurisation d'une application web, la génération de documents (PDF, QR code) et la conception d'interfaces responsives avec Bootstrap.

Annexe A — Script SQL de migration

Extrait essentiel de `schema_migration.sql` (les sections facultatives sont fournies en commentaires dans le fichier d'origine).

```
CREATE TABLE IF NOT EXISTS carte_etudiant (
    id              BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    idEtu           VARCHAR(20)     NOT NULL,
    date_generation DATETIME        NOT NULL DEFAULT CURRENT_TIMESTAMP,
    hash_qr         CHAR(64)        NOT NULL,
    chemin_pdf      VARCHAR(255)    NOT NULL,
    verifie_le      DATETIME        NULL,
    PRIMARY KEY (id),
    KEY idx_carte_etudiant (idEtu),
    KEY idx_carte_hash (hash_qr),
    CONSTRAINT fk_carte_etudiant
        FOREIGN KEY (idEtu) REFERENCES etudiant (idEtu)
        ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE = InnoDB DEFAULT CHARSET = utf8mb4 COLLATE = utf8mb4_unicode_ci;

-- Facultatif : photo de l'étudiant utilisée par la carte
-- ALTER TABLE etudiant ADD COLUMN photo VARCHAR(255) NULL;
```

Annexe B — Extraits de code significatifs

Signature et vérification d'un QR code (functions.php)

```
function token_carte(string $idEtu): string
{
    return hash_hmac('sha256', $idEtu, CLE_SECRETE);
}

function token_valide(string $idEtu, string $token): bool
{
    return hash_equals(token_carte($idEtu), $token);   // comparaison en temps constant
}
```

Protection CSRF (functions.php)

```
function csrf_token(): string
{
    if (empty($_SESSION['csrf'])) { $_SESSION['csrf'] = bin2hex(random_bytes(32)); }
    return $_SESSION['csrf'];
}

function verifier_csrf(): void
{
    $recu = (string)($_POST['csrf'] ?? '');
    if ($recu === '' || !hash_equals(csrf_token(), $recu)) {
        http_response_code(419);
        flash('danger', 'Session expirée ou jeton de sécurité invalide.');
        rediriger('index.php');
    }
}
```

Validation de la date de naissance (save.php)

```
$date = DateTimeImmutable::createFromFormat('Y-m-d', $donnees['dateNaiss']);
$valide = $date && $date->format('Y-m-d') === $donnees['dateNaiss'];
if (!$valide) {
    $erreurs[] = 'La date de naissance est invalide (format attendu : AAAA-MM-JJ).';
} else {
    $age = $date->diff(new DateTimeImmutable('today'))->y;
    if ($date > new DateTimeImmutable('today')) { /* date future */ }
    elseif ($age < 15 || $age > 80)            { /* âge incohérent */ }
}
```

Correction proposée pour la recherche live (app.js)

```
// Au lieu de concaténer du HTML, construire les nœuds avec textContent
var lien = document.createElement('a');
lien.className = 'list-group-item list-group-item-action';
lien.href = 'form.php?id=' + encodeURIComponent(e.idEtu);
var fort = document.createElement('strong'); fort.textContent = e.nomEtu;
lien.append(fort, ' ' + e.prenomEtu + ' — ' + e.idEtu);
zone.appendChild(lien);
```

Annexe C — Glossaire

Terme	Définition
CRUD	Create, Read, Update, Delete : les quatre opérations de base sur des données.
PDO	PHP Data Objects : interface d'accès aux bases de données avec requêtes préparées.
Requête préparée	Requête SQL dont les valeurs sont transmises séparément du texte, ce qui neutralise l'injection SQL.
CSRF	Cross-Site Request Forgery : action déclenchée à l'insu de l'utilisateur depuis un autre site ; contrée par un jeton secret.
XSS	Cross-Site Scripting : injection de code dans une page affichée à d'autres utilisateurs ; contrée par l'échappement des sorties.
CSP	Content-Security-Policy : en-tête qui limite les sources de scripts, styles et images qu'un navigateur peut charger.
HMAC-SHA256	Signature calculée avec une clé secrète : seul le détenteur de la clé peut produire une signature valide.
CR80	Format standard des cartes (85,6 × 54 mm), identique à celui des cartes bancaires.
PRG	Post / Redirect / Get : redirection après un POST pour éviter la double soumission.
Flash (message)	Message stocké en session, affiché une seule fois à la page suivante.
Dompdf	Bibliothèque PHP qui convertit du HTML/CSS en PDF.